Strict log-concavity of the normalized Bell function

A computer-assisted proof of MathDB problem 324014

Principia Math In Progress

September 12, 2026

Task 1: solve the original problem. For real $x > 0$, put

$$B(x) = e^{-1} \sum_{k=1}^{\infty} \frac{k^x}{k!}.$$

We prove Conjecture 5.4 of Chen–Guo–Wang [1, p. 649]. The argument treats every real $x \geq 1$. Its compact-range part uses rigorous interval arithmetic and explicit infinite-tail bounds; the remaining unbounded range is handled analytically.

Theorem 1. *The function $B(x)^{1/x}$ is strictly log-concave on $[1, \infty)$. More precisely,*

$$\left(\frac{\log B(x)}{x}\right)'' < 0 \qquad (x \geq 1).$$

Consequently the sequence $\{B_n^{1/n}\}_{n \geq 1}$ is strictly log-concave.

1 A curvature identity

Let $L(x) = \log B(x)$ and give the positive integers the probability distribution

$$p_x(k) = \frac{k^x/k!}{\sum_{j \geq 1} j^x/j!}.$$

All differentiated series converge locally uniformly, by the ratio test after multiplication by any fixed power of $\log k$. Thus

$$L'(x) = \mathbb{E}_x \log K, \qquad L''(x) = \mathrm{Var}_x(\log K).$$

Define

$$G(x) = x^2 L''(x) - 2xL'(x) + 2L(x), \qquad H(x) = xL'(x) - L(x). \tag{1}$$

Then

$$\left(\frac{L(x)}{x}\right)'' = \frac{G(x)}{x^3}, \qquad G(x) = x^2 \,\mathrm{Var}_x(\log K) - 2H(x). \tag{2}$$

We show $G(x) < -1/4$ for $1 \leq x \leq 10^6$, and $G(x) < -x/5$ for $x \geq 10^6$.

2 An elementary estimate on the unbounded range

Throughout this section $x \geq 10^6$. Let $r > 1$ be the unique solution of $r \log r = x$, and set

$$W = \log r, \quad s = \frac{r}{\sqrt{x}}, \quad F(t) = x \log t - t \log t + t, \quad A = \frac{e^{F(r)}}{\sqrt{2\pi r}}.$$

We will use

$$W > 11, \qquad s > 80. \tag{3}$$

Indeed $11e^{11} < 10^6 < 12e^{12}$, so $11 < W < 12$ at $x = 10^6$ and $s > 1000/12 > 80$ there. Both W and $s = e^{W/2}/\sqrt{W}$ increase subsequently. Write

$$a_k = \frac{k^x/k!}{A}, \quad M_0 = \sum_{k\ge 1} a_k, \quad M_2 = \sum_{k\ge 1} a_k \left(\log \frac{k}{r}\right)^2.$$

Lemma 2. *For $x \ge 10^6$,*

$$2.17s < M_0 < 3s, \qquad M_2 < \frac{3.42s + 0.01}{x}, \qquad \mathbb{E}_x \left(\log \frac{K}{r}\right)^2 < \frac{1.6}{x}. \tag{4}$$

All decimals in this proof denote exact terminating rational numbers.

Proof. Stirling's bounds [2] give, for integers $k \ge 1$,

$$a_k = \sqrt{\frac{r}{k}}\, \exp\{F(k) - F(r) - \epsilon_k\}, \qquad 0 < \epsilon_k < \frac{1}{12k}. \tag{5}$$

Let $I = [19r/20, 21r/20]$. Since $F'(r) = 0$ and $-F''(t) = x/t^2 + 1/t$, on I we have

$$\frac{0.9}{s^2} < -F''(t) < \frac{1.21}{s^2}. \tag{6}$$

For the upper estimate, use $400/361 + 20/209 < 1.21$ and $W > 11$; for the lower, $400/441 > 0.9$ suffices. On I, $k > 76000$, and hence

$$0.97e^{-1.21(k-r)^2/(2s^2)} < a_k < 1.03e^{-0.9(k-r)^2/(2s^2)}. \tag{7}$$

The prefactors follow from $\sqrt{20/21}(1 - 0.001) > 0.97$ and $\sqrt{20/19} < 1.03$.

We use the elementary lattice-sum bound

$$\left| \sum_{k\in\mathbb{Z}} f(k - r) - \int_{\mathbb{R}} f(t)\, dt \right| \le \mathrm{TV}(f). \tag{8}$$

Here f is integrable and of bounded variation, with the lattice sum absolutely convergent. To prove (8), partition the real line into intervals $[k - r, k + 1 - r]$; the discrepancy on an interval is bounded by its variation. Summing proves the assertion. For a Gaussian the variation is 2; for $t^2 e^{-\lambda t^2/2}$ it is $8/(e\lambda)$.

Put $R = r/20$. The function $e^{-1.21t^2/(2s^2)}\mathbf{1}_{[-R,R]}(t)$ has variation 2. Its integral outside $[-R, R]$ before truncation is at most

$$\frac{40s}{1.21\sqrt{x}} e^{-1.21x/800} < 0.001s. \tag{9}$$

This follows, for example, from $\sqrt{x} \ge 1000$, $1.21x/800 > 4$, and $(40/1210)e^{-4} < 0.001$. All lattice points in this truncated interval have $k \ge 1$. Equations (7)–(9), $\sqrt{2\pi} > 2.5$, and $s > 80$ yield

$$\sum_{k\in I} a_k > 0.97 \left(\frac{2.5}{1.1} - 0.001 - \frac{2}{80}\right) s > 2.17s. \tag{10}$$

Similarly, $\sqrt{2\pi} < 2.51$ and $\sqrt{0.9} > 0.94$ give

$$\sum_{k\in I} a_k < 1.03 \left(\frac{2.51}{0.94} + \frac{2}{80}\right) s < 2.8s. \tag{11}$$

On I, the mean value theorem gives $|\log(k/r)| \le 20|k - r|/(19r)$. Apply (8) to $t^2 e^{-0.9t^2/(2s^2)}$. Since $0.9^{3/2} > 0.85$ and $e > 2.7$,

$$\begin{aligned} \sum_{k\in I} a_k \left(\log \frac{k}{r}\right)^2 &< \frac{1.03(20/19)^2}{r^2} \left(\frac{\sqrt{2\pi}}{0.9^{3/2}} s^3 + \frac{8}{0.9e} s^2\right) \\ &< \frac{1.03(20/19)^2 s^3}{r^2} \left(\frac{2.51}{0.85} + \frac{8}{0.9 \cdot 2.7 \cdot 80}\right) < \frac{3.42s}{x}. \end{aligned} \tag{12}$$

We now bound both omitted tails explicitly. For $1 \le k < 19r/20$, $-F''(t) \ge x/r^2$ on $[k, r]$, so

$$F(k) - F(r) \le -\frac{x}{800} < -\frac{x}{1000}.$$

Equation (5), the number of such integers being less than r, gives left-tail mass at most $r^{3/2}e^{-x/1000}$ and left-tail second logarithmic moment at most $r^{3/2}W^2e^{-x/1000}$. As $r < x$ and $(\log x)^2 < x$ for $x \ge 10^6$, these are bounded by $x^{3/2}e^{-x/1000}$ and $x^{5/2}e^{-x/1000}$, respectively.

At $T = 21r/20$, (6) gives

$$F(T) - F(r) \le -\frac{9x}{8000} < -\frac{x}{1000}, \qquad F'(T) \le -\frac{9W}{200}.$$

Concavity therefore bounds the right-tail weights by $e^{-x/1000}e^{-(9W/200)(k-T)}$. Enumerate the integers $k > T$ as $k = T + z + m$, where $0 < z \le 1$ and $m = 0, 1, \ldots$. Since $e^{-9W/200} < 2/3$ and $0 < \log(k/r) \le (k-r)/r \le m+2$, the right-tail mass is at most $3e^{-x/1000}$ and its second logarithmic moment is at most

$$e^{-x/1000} \sum_{m \ge 0} (m+2)^2 (2/3)^m = 66e^{-x/1000}.$$

Thus the combined tail mass is at most $2x^2e^{-x/1000}$ and its second moment is at most $2x^3e^{-x/1000}$. Both are less than $0.01/x$: indeed

$$2x^4e^{-x/1000} < 0.01 \qquad (x \ge 10^6). \tag{13}$$

The left side decreases on this range; at $x = 10^6$ the inequality follows from $\log 200 < 6$, $\log 10^6 < 14$, and $6 + 4 \cdot 14 < 1000$. Combining these estimates with (10)–(12) proves the first two claims of (4). Finally

$$\frac{M_2}{M_0} < \frac{3.42 + 0.01/80}{2.17x} < \frac{1.6}{x},$$

which proves the last claim. □

Lemma 3. *For $x \ge 10^6$, $G(x) < -x/5$.*

Proof. By (4) and $3/\sqrt{2\pi W} < 1$,

$$B(x) = e^{-1}AM_0 < e^{F(r)-1}, \qquad L(x) < F(r) - 1.$$

Cauchy–Schwarz and (4) give $L'(x) > \log r - \sqrt{1.6/x}$, and hence

$$H(x) > x - r - \sqrt{1.6x} + 1 > \left(1 - \frac{1}{11} - \frac{1}{500}\right)x + 1 > 0.9x.$$

We used $x \ge 10^6 > 1.6 \cdot 500^2$. Also $\mathrm{Var}_x(\log K) \le \mathbb{E}_x(\log(K/r))^2 < 1.6/x$. Equation (2) now gives $G(x) < 1.6x - 1.8x = -x/5$. □

3 A rigorous certificate for the compact range

Lemma 4. *For every real $x \in [1, 10^6]$, $G(x) < -1/4$.*

The proof of this lemma is the finite interval computation below. We give its formulas, all truncation bounds, the exact cover, and the complete executable code in the appendix.

For any positive integer k_0, let

$$t_k = \log(k/k_0), \quad v_k = \exp\{xt_k + \log(k_0!) - \log(k!)\}, \quad S_j = \sum_{k \ge 1} v_k t_k^j \quad (j = 0, 1, 2).$$

Direct substitution in (1) gives the cancellation-resistant identity

$$G = x^2\left(\frac{S_2}{S_0} - \left(\frac{S_1}{S_0}\right)^2\right) - 2x\frac{S_1}{S_0} - 2\log(k_0!) + 2\log S_0 - 2. \tag{14}$$

For an exact rational interval $[a, b]$, the code chooses integers $1 \le \ell \le k_0 \le h$ and encloses the sums over $[\ell, h]$ using Arb real balls. Window selection uses floating point but is not relied upon for an inequality: the following two ratios are rigorously checked to be less than 1.

If $\ell > 1$, let

$$\rho_L = \ell(1 - 1/\ell)^a.$$

The ratio $v_{k-1}/v_k = k(1 - 1/k)^x$ increases with k and decreases with x. Thus the left omitted mass is at most

$$T_L = v_\ell^+ \frac{\rho_L}{1 - \rho_L}. \tag{15}$$

Here a superscript + means an outward upper bound valid on $[a, b]$. Its contributions to S_0, S_1, S_2 lie, respectively, in $[0, T_L]$, $[-T_L \log k_0, 0]$, and $[0, T_L(\log k_0)^2]$. For $\ell = 1$ the left tail is empty.

For the right tail put

$$\rho_R = \frac{(1 + 1/h)^b}{h + 1}, \quad c = \log(h/k_0), \quad R_0 = \frac{\rho_R}{1 - \rho_R}, \quad R_1 = \frac{\rho_R}{(1 - \rho_R)^2}, \quad R_2 = \frac{\rho_R(1 + \rho_R)}{(1 - \rho_R)^3}.$$

The adjacent ratio v_{k+1}/v_k decreases with k and increases with x. Furthermore $\log((h + m)/k_0) \le c + m/h$ for $m \ge 1$. The right contributions therefore lie in the nonnegative intervals with upper endpoints

$$v_h^+ R_0, \qquad v_h^+(cR_0 + R_1/h), \qquad v_h^+(c^2 R_0 + 2cR_1/h + R_2/h^2). \tag{16}$$

These enclose the entire infinite sums, including all logarithmic moments.

The initial closed intervals have endpoints $i^2/64$, $i = 8, \ldots, 8000$. An interval is accepted only if Arb proves that the upper bound from (14)–(16) is strictly less than $-1/4$; otherwise it is bisected at its exact rational midpoint. At 128-bit precision the run accepts **8001 intervals**, using **9 bisections**. It then checks, in exact rational arithmetic, that the intervals form a contiguous cover from 1 to 10^6. The complete record is `curvature-certificate-million.json`. This proves Lemma 4.

Proof of Theorem 1. Lemmas 3 and 4 cover $[1, \infty)$. Equation (2) proves the stated strict differential inequality. Strict concavity, restricted to successive positive integers, gives $2\log(B_n^{1/n}) > \log(B_{n-1}^{1/(n-1)}) + \log(B_{n+1}^{1/(n+1)})$ for $n \ge 2$. □

4 Verification and scope

The compact-range proof was run with Python 3.13.14 and `python-flint` 0.9.0. Reproduce it with

```
python certify_curvature.py --end 1000000 --prec 128 \
    --output curvature-certificate-million.json
python check_analytic_constants.py
```

The second command verifies the 18 elementary constant comparisons used above; their formulas are already included in the proof. The first command is an essential part of the proof, not a sample-point experiment. It relies on the enclosure guarantees of Arb, accessed through `python-flint` [3, 4]. The source code and both JSON records accompany this document.

Current eligibility checks, including the misleading MathDB reference annotation, are documented separately in `literature-audit.md`. This is a locally checked computer-assisted proof; no external referee assessment is claimed.

5 Follow-up tasks

Task 2: determine the full positive domain. Extend strict log-concavity of $B(x)^{1/x}$ to $0 < x < 1$, or determine the maximal interval of positive real arguments on which it holds.

Task 3: obtain a proof using analysis alone. Replace the compact-range certificate by a direct inequality between $\mathrm{Var}_x(\log K)$ and $xL'(x) - L(x)$ that is valid for every $x \geq 1$.

Task 4: classify the Poisson parameter. For $B_\lambda(x) = e^{-\lambda} \sum_{k \geq 1} k^x \lambda^k / k!$, determine all $\lambda > 0$ for which $B_\lambda(x)^{1/x}$ is strictly log-concave for every $x \geq 1$.

Task 5: obtain sharp curvature estimates. Derive a uniform expansion of $G(x)$ in terms of the positive solution $We^W = x$, with an explicit remainder, and determine optimal constants in two-sided bounds for $-(L(x)/x)''$.

References

[1] W. Y. C. Chen, J. J. F. Guo and L. X. W. Wang, Zeta functions and the log-behavior of combinatorial sequences, *Proc. Edinburgh Math. Soc.* 58 (2015), 637–651. doi:10.1017/S0013091515000036. Conjecture 5.4, p. 649; author manuscript arXiv:1208.5213.

[2] H. Robbins, A remark on Stirling's formula, *Amer. Math. Monthly* 62 (1955), 26–29. doi:10.2307/2308012.

[3] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Trans. Computers* 66 (2017), 1281–1292. arXiv:1611.02831.

[4] The python-flint contributors, *python-flint documentation: real balls*, `https://python-flint.readthedocs.io/en/latest/arb.html`.

A Complete compact-range certifier

```
"""Rigorous real-interval certificate, with geometric bounds for both tails.

Run: python certify_curvature.py
Requires python-flint. Floating point is used only to select a summation window;
all claims, all domain endpoints, and all tail estimates use Arb or exact rationals.
"""
import argparse
import json
import math
import time
from pathlib import Path
from flint import arb, fmpq, ctx

parser = argparse.ArgumentParser()
parser.add_argument("--end", type=int, default=100000)
parser.add_argument("--prec", type=int, default=128)
parser.add_argument("--output", default="curvature-certificate.json")
args = parser.parse_args()
ctx.prec = args.prec
logs = [arb(0)]
facts = [arb(0)]


def extend(n):
    for k in range(len(logs), n + 1):
        logs.append(arb(k).log())
        facts.append(facts[-1] + logs[-1])


def nonnegative_interval(bound):
    assert bound >= 0
    return arb(0).union(bound.upper())


def choose_window(a, b):
    mid = (float(a) + float(b)) / 2
    w = max(0.5, math.log1p(mid))
    for _ in range(12):
        w -= (w + math.log(w) - math.log(mid)) / (1 + 1/w)
    r = mid / w
    k0 = max(1, round(r))
    width = math.ceil(16*r/math.sqrt(mid) + 40 + float(b-a))
    return k0, max(1, k0-width), k0+width


def certify(a, b):
    k0, lo, hi = choose_window(a, b)
    extend(hi + 1)
    x = arb((a+b)/2, (b-a)/2)
    t0 = logs[k0]
    s0, s1, s2 = arb(0), arb(0), arb(0)
    vl = vr = None
    for k in range(lo, hi+1):
        t = logs[k] - t0
        v = (x*t + facts[k0] - facts[k]).exp()
        s0 += v
        s1 += v*t
        s2 += v*(t*t)
        if k == lo:
            vl = v
        if k == hi:
            vr = v

    # For k <= lo, w[k-1]/w[k] <= lo*(1-1/lo)^a.
    if lo > 1:
        rho = arb(lo) * (arb(a)*(logs[lo-1]-logs[lo])).exp()
        assert rho < 1
        mass = vl.upper() * rho / (1-rho)
        s0 += nonnegative_interval(mass)
        s1 -= nonnegative_interval(mass*t0)
        s2 += nonnegative_interval(mass*t0**2)

    # For k >= hi, w[k+1]/w[k] <= (1+1/hi)^b/(hi+1).
    rho = (arb(b)*(logs[hi+1]-logs[hi])).exp() / (hi+1)
    assert rho < 1
    r0 = rho/(1-rho)
    r1 = rho/(1-rho)**2
    r2 = rho*(1+rho)/(1-rho)**3
    c = logs[hi]-t0
```

```
    vh = vr.upper()
    s0 += nonnegative_interval(vh*r0)
    s1 += nonnegative_interval(vh*(c*r0+r1/hi))
    s2 += nonnegative_interval(vh*(c*c*r0+2*c*r1/hi+r2/(hi*hi)))
    assert s0 > 0
    mean = s1/s0
    g = x*x*(s2/s0-mean*mean)-2*x*mean-2*facts[k0]+2*s0.log()-2
    return g, (lo, hi, k0)


start = time.monotonic()
records = []
pending = []
previous = fmpq(1)
for i in range(9, math.ceil(8*math.sqrt(args.end))+1):
    endpoint = min(fmpq(i*i, 64), fmpq(args.end))
    if endpoint > previous:
        pending.append((previous, endpoint))
        previous = endpoint
assert previous == args.end
pending.reverse()
splits = 0
last_report = start
while pending:
    a, b = pending.pop()
    g, window = certify(a, b)
    if g < -arb(fmpq(1,4)):
        mid = [int(z) for z in g.mid().man_exp()]
        rad = [int(z) for z in g.rad().man_exp()]
        assert arb(tuple(mid), tuple(rad)) < -arb(fmpq(1,4))
        records.append(dict(a=str(a), b=str(b), g_enclosure=g.str(30, more=True),
                            g_mid_dyadic=mid, g_rad_dyadic=rad,
                            summation_window=list(window)))
    else:
        assert b-a > fmpq(1, 2**30), "Interval subdivision exhausted"
        middle = (a+b)/2
        pending.append((middle, b))
        pending.append((a, middle))
        splits += 1
    now = time.monotonic()
    if now-last_report >= 15:
        print(json.dumps(dict(certified_intervals=len(records), splits=splits,
                              through=records[-1]["b"] if records else None,
                              pending=len(pending), seconds=round(now-start,1))),
              flush=True)
        last_report = now

# Independently check the exact interval cover before accepting the run.
cursor = fmpq(1)
for rec in records:
    assert fmpq(rec["a"]) == cursor
    cursor = fmpq(rec["b"])
    assert arb(tuple(rec["g_mid_dyadic"]), tuple(rec["g_rad_dyadic"])) < -arb(fmpq(1,4))
assert cursor == args.end
out = dict(problem=324014, domain=["1",str(args.end)], precision_bits=args.prec,
           conclusion="g(x) < -1/4 on every listed closed interval",
           interval_count=len(records), subdivisions=splits,
           seconds=round(time.monotonic()-start,3), intervals=records)
Path(__file__).with_name(args.output).write_bytes(
    (json.dumps(out,indent=2)+"\n").encode("utf-8"))
print(json.dumps({k:v for k,v in out.items() if k != "intervals"}), flush=True)
```